

PRISM — Đặc tả Pipeline (Spec toàn repo)

commit 670d68a · scope: prism/src/prism, prism/scripts, prism/kaggle_notebooks · quy ước vị trí theo tên hàm/module, không dùng số dòng

PRISM đo **sự thay đổi theo thời gian của sentiment ở mức aspect** trong review khách sạn Booking.com. Chuỗi giá trị: dùng gold train một extractor mT5 → chạy extractor trên pool review chưa gán nhãn → ước lượng độ tin cậy từng quad (cross-modal reliability) → phát hiện temporal drift có hiệu chỉnh thành phần & mùa vụ. Tài liệu này đặc tả **từng step**: nhiệm vụ, input, output, tham số và logic cốt lõi. Máy chạy: **GPU** **CPU**

NET (cần mạng) **TÙY CHỌN**

0. Tổng quan kiến trúc & thứ tự bất biến

Thứ tự chạy **bất biến** (không đảo được vì phụ thuộc dữ liệu):

A store → B-data → B-train → B-selftrain → B-infer → (photos) → C(train_verifier→apply_verifier→bridge→apply) → D drift → [injection]

Thư mục output (theo config.py)

Hằng	Đường dẫn mặc định
STORE_DIR	outputs/store — Module A
EXTRACT_DIR	outputs/extract — Module B
RELIAB_DIR	outputs/reliability — Module C
DRIFT_DIR	outputs/drift — Module D
MODEL_DIR	models — checkpoint

Biến môi trường override

Env	Ý nghĩa
PRISM_TABSA_ROOT	gốc repo
PRISM_WORK_DIR	gốc outputs/
PRISM_MODEL_DIR	gốc models/
PRISM_HAMOS_ROOT	gốc gold quads/ảnh hamos-mabsa/

Sơ đồ phụ thuộc

```
data/raw + hamos-mabsa
├─(A)─ module_a_store ─────────→ store/{reviews.jsonl.gz, hotel_cohorts.json}
│                                     | (store là input của selftrain & infer)
├─(B-data)─ module_b_data ───→ *.t2t.jsonl
│                                     (B-train) ─→ models/seed_extractor
│                                     (B-selftrain) ← seed + store ─→ models/selftrain_round* (final_ckpt)
│                                     (B-infer) ← final_ckpt + store ─→ extract/pool_quads.<cohort>.jsonl.gz
│                                     (photos) ─→ reliability/pool_image_index.json
│                                     (C) train_verifier→apply_verifier→bridge→apply ─→ reliability/quads_weighted.jsonl.gz
│                                     (D) module_d_drift ─────────→ drift/drift_results.*.json
```

1. Dữ liệu đầu vào (tiên quyết)

Item	Đường dẫn	Vai trò
Pool chưa nhãn	data/raw/hotel_booking_unlabeled.jsonl	~1,95M review thô cho Module A
Gold split	data/raw/{train,dev,test}.jsonl	train/val/test extractor; hotel-disjoint
Gold ABSA meta	data/raw/hotel_absa_labeled.jsonl	bổ sung review_date (không chứa quad)
Gold quads/seg/ảnh	hamos-mabsa/ (qua PRISM_HAMOS_ROOT)	quad gold + ảnh gold cho B-data & C

Mỗi dòng gold: source_review_id, segment_id, text, review_date, quads[] . Mỗi quad = (aspect_term, taxonomy_code, aspect_category, opinion_term, sentiment) + span.

2. Module A — Temporal Quad Store CPU

src/prism/module_a_store.py · python -m prism.module_a_store

Biến pool thô thành corpus sạch, chống leakage gold, gán nhãn thời gian/cohort/stratum

INPUT data/raw/hotel_booking_unlabeled.jsonl, {train,dev,test}.jsonl, gold ABSA meta

OUTPUT outputs/store/reviews.jsonl.gz, hotel_cohorts.json, store_report.json

LOGIC A1 parse ngày (regex tiếng Việt) · A2 cắt cửa sổ 2022-03 → 2025-02 (36 kỳ tháng, bỏ 2 biên dữ liệu dờ) · A3 dedup · **A4 GOLD BLOCKLIST [BLOCKING]** · A5 ép kiểu hotel_id · A6 lọc/cờ text-bearing (tách text_pos / text_neg) · A7 langid (tùy chọn, cần fasttext) · A8 gán stratum + cohort . **Blocklist** 3 khóa chặn gold lọt vào pool: (1) (hotel_id, iso_date) ; (2) hash 120 ký tự đầu text; (3) map hotel → gold_split (để gán cohort T-unbiased) .

Schema mỗi dòng `reviews.jsonl.gz`

`review_uid · hotel_id · period · stratum · text_pos · text_neg · score · has_photo · n_words · in_gold`. Cờ `in_gold=true` = BLOCKLIST, không bao giờ được suy luận.

3. Module B — Extractor (mT5)

3.1 B-data — Text-to-Text CPU

`src/prism/module_b_data.py · python -m prism.module_b_data`

Chuyển gold (review → quads) thành cặp text2text cho mT5, kèm parser đảo ngược

INPUT	gold quads (<code>hamos-mabsa/</code>), <code>data/raw/{train,dev,test}.jsonl</code>
OUTPUT	<code>outputs/extract/{train,dev,test}.t2t.jsonl</code> , <code>chrono_{train,test}.t2t.jsonl</code> , <code>data_report.json</code>
LOGIC	Prefix input <code>"extract quads: "</code> . Target tuyến tính hóa: <code><quad> aspect taxonomy_code opinion sentiment </quad> ...</code> — aspect/opinion implicit = token <code>NULL</code> . <code>parse_linearized()</code> là nghịch đảo, dùng khi infer; hard-filter taxonomy ngay tại parse (bỏ quad có <code>code#CODE2CAT</code> hoặc <code>sentiment</code> lạ). Tạo thêm chronological split (probe E1c).

3.2 B-train — Seed extractor GPU

`src/prism/module_b_train.py · python -m prism.module_b_train --model google/mt5-small --epochs 10`

Fine-tune mT5 trên gold train → seed teacher

INPUT	<code>extract/train.t2t.jsonl</code> , <code>extract/dev.t2t.jsonl</code> , base <code>google/mt5-small</code> (docstring mặc định <code>mt5-base</code> ; Kaggle dùng <code>small</code>)
OUTPUT	<code>models/seed_extractor/</code> (<code>config.json</code> + <code>model.safetensors</code> + tokenizer + <code>training_log.json</code>)
THAM SỐ	<code>--epochs 10 · --batch · --grad-accum · --lr 3e-4 · --max-src 160 · --max-tgt 192 · --seed</code> . Tự chọn device <code>cuda/mps/cpu</code> . mT5 vì gold đa ngữ (80,5% en · 16,2% vi).

3.3 B-eval — Đánh giá GPU TỰY CHỌN

`src/prism/module_b_eval.py · python -m prism.module_b_eval`

Đo precision / recall / F1 của prediction trên dev/test

INPUT	checkpoint + <code>{dev,test}.t2t.jsonl</code>
OUTPUT	<code>outputs/extract/eval_report.json</code>
LOGIC	Phân tích theo ngôn ngữ, độ dài, polarity, category. Không nằm trong chuỗi Kaggle bắt buộc.

3.4 B-selftrain — Teacher→Student GPU

`src/prism/module_b_selftrain.py · python -m prism.module_b_selftrain --rounds 2 --cohort B-anchor`

Vòng lặp self-training có kiểm soát: pseudo-label pool → train student → early-stop theo dev F1 & phân phối lớp

INPUT	<code>models/seed_extractor/</code> , <code>extract/{train,dev}.t2t.jsonl</code> , <code>data/raw/dev.jsonl</code> (gold eval), store (<code>reviews.jsonl.gz</code> + <code>hotel_cohorts.json</code>)
OUTPUT	<code>models/selftrain_round<r>/</code> , <code>extract/selftrain_history.json</code> (chứa <code>final_ckpt</code>)
MỖI VÒNG	(1) gọi <code>module_b_infer</code> pseudo-label (mặc định <code>--infer-limit 200000</code> , loại <code>in_gold</code>) → (2) chọn pseudo: <code>conf_seq ≥ --tau (0.7)</code> VÀ <code>p_posterior ≥ --tau-post (0.8)</code> — lọc mềm, quad <code>provenance_flip</code> vẫn giữ nếu đủ posterior → (3) trộn gold u pseudo, giới hạn <code>--max-ratio 3.0</code> (pseudo/gold), train student → (4) đo dev macro-F1, DỪNG nếu dev giảm hoặc <code>%negative</code> pseudo lệch > <code>--max-skew 0.10</code> so với gold (15,4%).
LƯU Ý	Store là input của infer nội bộ , KHÔNG phải sản phẩm của B — nó là output Module A.

3.5 B-infer — Pool chính thức GPU

`src/prism/module_b_infer.py · python -m prism.module_b_infer --ckpt models/selftrain_round<N> --cohort <cohort>`

Chạy student cuối trên toàn store → sinh quad + provenance posterior

INPUT	checkpoint student cuối (<code>finalckpt</code>), store (<code>reviews.jsonl.gz</code> + <code>hotel_cohorts.json</code>)
OUTPUT	<code>outputs/extract/pool_quads.<cohort>.jsonl.gz</code> — lập cho <code>T-unbiased</code> · <code>A-dense</code> · <code>B-anchor</code> · <code>corpus</code>
LOGIC	Mỗi review → tối đa 2 đơn vị suy luận: (<code>text_pos</code> , <code>φ=POS</code>) & (<code>text_neg</code> , <code>φ=NEG</code>) — nguồn trường theo từng đoạn. Provenance posterior (không lọc cứng): $P(s x, \phi) \propto P_{\text{model}}(s x)^{\lambda} \cdot P(s \phi)^{(1-\lambda)}$, $\lambda=0.7$. $P_{\text{model}}(s x)$ là xác suất THẬT: rescore 3 biến thể sentiment mỗi quad bằng teacher forcing rồi softmax (dùng <code>--no-rescore</code> để về xấp xỉ one-hot khi cần tốc độ). Quad có <code>s_post ≠ s_model</code> → cờ <code>provenance_flip</code> (giữ, để audit).
THAM SỐ	<code>--batch</code> · <code>--score-batch</code> (batch rescore) · <code>--lam</code> · <code>--limit</code>

Schema mỗi quad `pool_quads.<cohort>.jsonl.gz`

Nhóm	Trường
Quad	<code>aspect_term</code> · <code>taxonomy_code</code> · <code>aspect_category</code> · <code>opinion_term</code> · <code>aspect_implicit</code> · <code>opinion_implicit</code>
Ngữ cảnh	<code>text</code> · <code>review_uid</code> · <code>hotel_id</code> · <code>period</code> · <code>stratum</code> · <code>phi</code> · <code>score</code> · <code>has_photo</code> · <code>n_words</code>
Suy luận	<code>conf_seq</code> (độ tin cậy seq) · <code>sentiment_model</code> · <code>p_model{pos, neu, neg}</code> · <code>sentiment</code> (posterior s*) · <code>p_posterior</code> · <code>provenance_flip</code>

4. Tải ảnh pool (cho Module C) NET

`scripts/download_pool_photos.py` · `python scripts/download_pool_photos.py --cohort <cohort> --limit 0`

Tải ảnh đầu tiên của mỗi review pool (has_photo, không in_gold, đúng cohort)

INPUT	<code>outputs/store/reviews.jsonl.gz</code>
OUTPUT	<code>outputs/reliability/pool_images/<review_uid>.jpg</code> + <code>pool_image_index.json</code> ({review_uid: đường dẫn tuyệt đối})
LOGIC	Resume an toàn (ảnh đã có bị bỏ qua). Rate-limit <code>--sleep 0.4s</code> , <code>--timeout 15s</code> . Vì index chứa đường dẫn tuyệt đối , photos & các step C nên chạy CÙNG session.

(kèm) Mẫu audit — `python -m prism.make_audit_samples`

OUTPUT `outputs/reliability/audit_sample_300.jsonl`, `d0_sample_300.jsonl` (dùng ở stage `bridge`)

5. Module C — Cross-Modal Reliability CPU / GPU

`src/prism/module_c_reliability.py` · lỗi novelty, có điểm GO/NO-GO · 4 stage đúng thứ tự

GO/NO-GO (chốt trước khi chạy): $AUC_{\text{verifier trên gold test}} \geq 0.70$; $AUC(V) - AUC(\text{baseline không ảnh}) \geq 0.05$ (chống rò rỉ category-prior); $Spearman(\hat{r}, \text{human audit}) \geq 0.30$. Trượt bất kỳ ngưỡng → **bỏ Module C**, dùng `w_q = conf_seq` đã temperature-scale (pipeline vẫn chạy, lùi về A+B+D).

Stage	Input	Output	Nhiệm vụ
<code>train_verifier</code> (C1)	gold ảnh <code>hamos-mabsa/</code>	<code>verifier.pkl</code> , <code>verifier_report.json</code>	Học $V(\text{image}, \text{category}) \rightarrow [0,1]$ trên 9.219 ảnh gold, mức REVIEW×CATEGORY. Split theo hotel.
<code>apply_verifier</code> (C2)	<code>pool_quads.<cohort></code> , <code>pool_image_index.json</code>	<code>pool_quads_vimg.<cohort>.jsonl.gz</code> (thêm <code>v_image</code>)	Áp V lên quad có ảnh đã tải → điểm ảnh hỗ trợ category.
<code>bridge</code> (C3)	<code>pool_quads_vimg.<cohort></code> (cần <code>v_image</code>), <code>audit_sample_300.jsonl</code>	<code>bridge.pkl</code> , <code>bridge_report.json</code>	Fit $g(\text{đặc trưng chỉ-text}) \approx P[V=1]$ (kèm IPW theo propensity has_photo) → áp cho MỌI quad kể cả không ảnh.
<code>apply</code>	<code>pool_quads.<cohort></code> + <code>bridge.pkl</code>	<code>quads_weighted.jsonl.gz</code> (thêm <code>w</code>)	$w_q = 1[\text{tax hợp lệ}] \cdot 1[\text{span hợp lệ}] \cdot \hat{r}(q)$.

Ghi chú nổi: `bridge` đọc file `vimg` (không phải `pool_quads` gốc) vì cần `v_image`; `apply` đọc `pool_quads` gốc + `bridge.pkl`.

6. Module D — Dual-Bias Compositional Drift CPU [kết quả cuối]

`src/prism/module_d_drift.py` · `python -m prism.module_d_drift --quads outputs/reliability/quads_weighted.jsonl.gz --cohort corpus --level taxonomy_code`

Phát hiện drift theo thời gian trên 2 kênh, mỗi kênh có bản raw & adjusted

INPUT	<code>quads_weighted.jsonl.gz</code> (hoặc <code>pool_quads.*</code> với <code>w=conf_seq</code> nếu bỏ Module C — cùng schema)
OUTPUT	<code>outputs/drift/drift_results.<cohort>.<level>.json</code>
HAI KÊNH	π prevalence — SHARE-OF-COMPLAINTS: tỷ trọng aspect trong tổng lượt phàn nàn (ϕ =NEG hoặc <code>sentiment=negative</code>), thang CLR. v valence — NEGATIVITY RATE: <code>P(negative nhắc aspect)</code> trên MQI quad, direct standardization theo strata. Đây là estimand trung tâm.
BƯỚC	hiệu chỉnh recall (nếu có <code>recall_table.json</code> từ D0) → direct standardization trên strata → khử mùa vụ (trừ TB tháng-trong-năm) → OLS trend + best-split changepoint → permutation null riêng từng chuỗi → BH-FDR (lưới adj = kết quả chính; lưới raw FDR riêng cho injection E3a) → bootstrap CI slope π -adj.
THAM SỐ	<code>--level taxonomy_code aspect_category</code> · <code>--cohort</code> . Thống kê: <code>N_BOOTSTRAP=1000</code> · <code>N_PERMUTATION=1000</code> · <code>FDR_ALPHA=0.05</code> · <code>T_THRESHOLD=2.5</code> · <code>SEED=20260821</code> .

7. Robustness — Injection & Negative Control CPU TÙY CHỌN

`src/prism/eval_injection.py` · `python -m prism.eval_injection --quads <file> --test {composition,valence,shuffle}`

Vì không có drift ground-truth, tạo drift/không-drift có kiểm soát trên dữ liệu thật rồi kiểm pipeline trả lời đúng

INPUT	<code>quads_weighted.jsonl.gz</code>
OUTPUT	<code>outputs/drift/injection-<test>.json</code> (không đề <code>drift_results</code> chính)
TEST	E3a composition : shuffle timestamp (nền null) rồi resample tỷ trọng strata trôi dần → PASS nếu adj KHÔNG báo drift & raw CÓ báo (cùng máy suy diễn). E3c valence : sau t0 lật 8% quad 1 aspect pos→neg (không đổi ϕ) → v-adj phải bắt, đo theo <code>6∈{2,5,10,20}%</code> . E4 shuffle : xáo timestamp toàn bộ, lặp <code>--repeats</code> → FPR $\approx \alpha$.

Kèm: `scripts/probe_complaint_composition.py` — probe keyword độc lập kiểm chứng complaint composition (không bắt buộc).

8. Orchestrator Kaggle — `scripts/kaggle_pipeline.py`

Gộp toàn bộ `src/prism` thành 1 entripoint, mỗi cell = 1 step để lỗi step nào chỉ hỏng step đó. Tự tìm input theo tên file đệ quy khắp `/kaggle/input` + `/kaggle/working` (kể cả output notebook đã attach), stage vào `WORK_DIR/MODEL_DIR`; output ghi ở `/kaggle/working` . Thiếu file nào báo lỗi rõ "chạy step X trước".

Step: `store` · `data` · `train` · `selftrain` · `infer` · `photos` · `c_verifier` · `c_apply_verifier` · `c_bridge` · `c_apply` · `drift` · `injection` · `all` . Với `--step all` : chạy tuần tự, BỎ QUA step đã có output (dựa vào `sentinel()`); `--force` để chạy lại.

```
# Cell setup
!rm -rf /kaggle/working/Prism
!git clone --depth 1 https://github.com/hotuyen21pt/Prism.git /kaggle/working/Prism
P = "/kaggle/working/Prism/prism/scripts/kaggle_pipeline.py"

# mỗi cell 1 step
!python {P} --step train
!python {P} --step selftrain --cohort B-anchor --infer-batch 2 --infer-score-batch 4
!python {P} --step infer --cohort T-unbiased --batch 2 --score-batch 4 \
    --ckpt /kaggle/input/.../pipeline-selftrain/models/selftrain_round2
!python {P} --step photos --cohort T-unbiased
!python {P} --step c_verifier
!python {P} --step c_apply_verifier --cohort T-unbiased
!python {P} --step c_bridge --cohort T-unbiased
!python {P} --step c_apply --cohort T-unbiased
!python {P} --step drift --cohort corpus
```

Cạm bẫy vận hành đã gặp

- infer báo thiếu `hotel_cohorts.json`** : chưa attach output `store` (Module A) vào notebook infer. Attach dataset store + **Save Version** (attach tạm trong editor không lưu vào bản commit).
- Auto-detect checkpoint chọn sai**: khi có nhiều checkpoint attach, `find_ckpt()` ưu tiên nhầm thư mục tên `seed_extractor` thay vì `selftrain_round2` . Luôn truyền `--ckpt` tay tới student cuối.
- photos** → `c_*` nên chạy CÙNG session vì `pool_image_index.json` chứa đường dẫn tuyệt đối.

9. Tham chiếu — Cohort · Taxonomy · Strata

Cohort (`COHORT_DEFS`)

Taxonomy

Cohort	Định nghĩa
A-dense	needs_gold, min_pool 1000 — 276 hotel
B-anchor	needs_gold, min_pool 300 — 1.221 hotel
T-unbiased	gold_split=test — 514 hotel (extractor chưa thấy)
corpus	toàn bộ 10.631 hotel

6 category: FACILITY · AMENITY · SERVICE · EXPERIENCE · LOYALTY · BRANDING . 14 code CODES_REPORTABLE (≥300 quad gold) báo cáo riêng; 17 code hiếm gộp lên category cha (CODE2CAT). Sentiment: positive · neutral · negative .

Strata (Module D) & ngưỡng

COUNTRY_BLOCS = VN/WEST/ASIA/OTH · TRAVELLER_TYPES (Cặp đôi 40,8% · Gia đình 24,0% · Khách lẻ 18,9% · Nhóm 16,0%) · LENGTH_BINS L0/L1/L2 (mặc định USE_LENGTH_IN_STRATA=False → 20 ô). MIN_STRATUM_N=30 · MIN_VALENCE_CELL_W=10 . Provenance prior POS(.931/.036/.033) NEG(.217/.142/.641) , λ=0.7 .

10. Test & verification

```
$env:PYTHONPATH="src"
python -m unittest discover tests          # test_utils · test_module_b_data · test_module_d · test_injection
python -m prism.smoke_test                 # smoke các phần không cần Torch
```

- **Bất biến leakage**: mọi dòng in_gold=true phải bị loại ở infer/selftrain — đừng bỏ blacklist khi refactor Module A.
- **Thứ tự selftrain → infer**: infer chính thức phải dùng student cuối; bản pool_quads sinh bằng seed teacher chỉ là sơ bộ.
- **Provenance mềm**: KHÔNG chuyển posterior thành bộ lọc cứng — quad provenance_flip vẫn giữ để audit.
- **Idempotency**: photos & --step all bỏ qua việc đã xong; muốn chạy lại dùng --force / xóa output.

PRISM pipeline spec · nguồn: docs/pipeline_io.md , docs/repo_flow.md , src/prism/*.py , scripts/kaggle_pipeline.py · commit 670d68a · vị trí trích theo tên module/hàm (số dòng có thể đổi).